05 機率分析與隨機演算法 • 獨立詳解

05-E01 依賴事件仍可加期望

推導

令I1,I2表示各骰是否6，X=I1+I2。每個邊際機率1/6，所以E[X]=1/6+1/6=1/3，這一步不需兩者獨立。恰一顆6是兩個互斥事件，其機率為(1/6)(5/6)+(5/6)(1/6)=5/18；把每項交集拆成乘積時用到獨立性。

若第二顆完全複製第一顆，兩個指示仍各期望1/6，E[X]仍1/3；但X只可能0或2，P(X=1)=0。獨立性改變聯合分布，沒有使期望線性性失效。

結論與檢核

獨立時E=1/3、恰一顆機率5/18；完全相關時E=1/3、恰一顆機率0。

05-E02 隨機排列的固定點

推導

Ii表示第i張名牌在自身位置，有(n−1)!個排列，P(Ii=1)=1/n，因此E[F]=n/n=1。固定兩個指定位置則剩(n−2)!排列，聯合機率1/[n(n−1)]，與各邊際乘積$1/n^2$不同，故不獨立。

n=1只有恆等排列，F=1；n=0只有空排列、F=0。兩固定點的公式僅在n≥2有意義。

結論與檢核

期望1不代表必恰有一個固定點，也不代表固定點指示獨立。n=2直接有F=0、2各半，和E=1一致。

05-E03 反向 Fisher–Yates 的六條路徑

推導

j2	j1	輸出
0	0	[b,c,a]
0	1	[c,b,a]
1	0	[c,a,b]
1	1	[a,c,b]
2	0	[b,a,c]
2	1	[a,b,c]

05 機率分析與隨機演算法 • 獨立詳解 （續）

各路徑機率1/3・1/2=1/6。一般第i輪前suffix i+1..n−1已固定；條件於任何已固定的suffix，剩餘i+1個元素各以1/(i+1)機率被放到位置i，之後不再改。對任何指定完整排列，連乘1/n、1/(n−1)、…、1/2給1/n!，每個選擇都有唯一位置可取。

結論與檢核

六種排列等機率；一般聯合分布亦均勻。這是正文正向版本的反向等價證明，不依賴「每個位置都均勻」的不足條件。

05-E04 隨機分割的期望訊息數

推導

每次p+(n−p)=n，直到所有葉大小1，葉大小總和仍n，故每次執行葉數恰n；期望Ln=n。也可由L1=1、Ln=2Σj<n Lj/(n−1)強歸納驗證。

星號S1=0，對p條件化後Sn=n+2Σj<n Sj/(n−1)。乘n−1，再與n+1式相減，得到nS(n+1)−(n−1)Sn=2n+2Sn，即S(n+1)=2+(n+1)Sn/n。令Qn=Sn/n便有Q(n+1)−Qn=2/(n+1)，Q1=0，故Qn=2(Hn−1)。

結論與檢核

葉數固定n；星號期望2n(Hn−1)。n=5為5個驚嘆號、77/6個星號的期望。後者不是實際印出分數個字元；不同執行的整數成本在取平均後可成分數。

05-E05 恰好兩次雇用

推導

一次恰為全域最佳在第一位，機率1/n。n次只有遞增順序一種，機率1/n!。兩次時，第二次雇用者必為全域最佳，設其位置i≥2；在此之前只雇第一位若且唯若前i−1的最大者在第一位。條件於最佳在i，其前綴相對順序仍均勻，機率1/(i−1)。

不同i互斥，所以P(X=2)=Σi=2..n 1/[n(i−1)]=H(n−1)/n。

結論與檢核

n=6依序1/6、H5/6=137/360、1/720。n=1時「一次」與「n次」是同一事件，不是互斥類；兩次不可能，須分開處理。

05 機率分析與隨機演算法 • 獨立詳解 （續）

05-E06 邊際均勻與隨機路徑數

推導

四個offset各以1/4抽到，對固定元素與目標位置恰有一個offset，故邊際均勻。但只會出現原序列的四種循環平移，其餘20種排列機率0，不能全為1/24。

三元素全範圍swap共有3^3=27條等機率選擇路徑。每個輸出機率是整數/27，而1/6不能如此表示，故至少有輸出偏離均勻。正確版本第i輪後前綴不再改動；全範圍swap後面的輪次能改回這些格，失去固定前綴條件。

結論與檢核

兩程序均不保證均勻排列。證明的是聯合分布失敗，不能靠看似混亂或單元素直方圖否定此反例。

05-E07 優先值平手的偏差

推導

16組等機率優先值中，P1<P2共6組，P1=P2共4組，P1>P2共6組。穩定平手令原順序機率10/16=5/8，逆序6/16=3/8。

整組重抽時接受率12/16=3/4；條件於不平手，6組給每一順序，所以各半。獨立重試接受的最終分布是此相同條件分布：Σr≥0(1/4)^r(6/16)=1/2。輪數尾和給1/(3/4)=4/3。

結論與檢核

穩定平手造成偏差；拒絕整組碰撞後恰均勻，期望4/3輪。不能把高機率無平手當成無條件精確均勻。

05-E08 非均勻生日的遷移

推導

獨立兩人的碰撞事件按共同日期互斥分解，單對機率q=Σr pr^2。k人碰撞對數期望C(k,2)q，線性性不需各對指示獨立。若共有n種日期，Σr$(pr-1/n)^2$=q−1/n≥0，所以q≥1/n，等號若且唯若每個pr=1/n。

題給q=1/4+1/16+1/64+1/64=11/32，三人期望33/32。全不同的機率是各三日期子集乘積乘3!：

$$6\left(\frac{1}{64}+\frac{1}{64}+\frac{1}{128}+\frac{1}{256}\right)=\frac{33}{128}.$$

05 機率分析與隨機演算法 • 獨立詳解 （續）

結論與檢核

有碰撞機率95/128，與期望33/32不同。超過1的期望不是機率，也不能套均勻日期的簡單乘積，因條件於已占日期後剩餘總機率依其身分而變。

05-E09 三桶四球與集滿等待

推導

每桶空機率(2/3)^4，空桶期望3・16/81=16/27。每桶恰一球機率C(4,1)(1/3)(2/3)^3，總期望32/27。全滿等價從4個具名投球到3桶的滿射，排容得$(3^4-3\cdot 2^4+3\cdot 1^4)/3^4=36/81=4/9$。

若改從0球投到全滿，三個幾何階段成功率1、2/3、1/3，期望1、3/2、3，總11/2。這裡投球總數是隨機變數，不固定4。

結論與檢核

固定四球：空桶期望16/27、單球桶期望32/27、全滿機率4/9；等待全滿期望11/2球。

05-E10 重疊視窗不是獨立試驗

推導

有6個長3視窗，每個全H機率1/8，故E[X]=6/8=3/4。對尚未出現HHH的前綴，記尾端連H恰0、1、2的數目(a,b,c)。追加T轉到0，追加H令0→1、1→2；2再接H便違規，不保留。更新為(a+b+c,a,b)。

已擲次數	a	b	c	合法總數
0	1	0	0	1
1	1	1	0	2
2	2	1	1	4
3	4	2	1	7
4	7	4	2	13
5	13	7	4	24
6	24	13	7	44
7	44	24	13	81
8	81	44	24	149

每個合法新前綴依最後符號與舊尾長唯一歸類，故遞迴不漏不重。256個等機率長8字串中149個無HHH。

05 機率分析與隨機演算法 • 獨立詳解 （續）

結論與檢核

P(L≥3)=107/256，不是3/4。一次長4連H會貢獻兩個長3視窗，E[X]在數所有視窗，事件機率只問至少一個。

05-E11 有限七人的線上門檻

推導

最佳在i>k時成功若且唯若前i−1最高者在前k位，條件機率k/(i−1)；最佳位置機率1/7。故Pk=(k/7)Σj=k..6 1/j。設qk為該和，P(k+1)−Pk=(q(k+1)−1)/7，且qk嚴格下降。

q2=29/20>1，q3=19/20<1，故最佳k=2，P2=29/70。其餘k=1、3、4、5、6成功率分別7/20、57/140、37/105、11/42、1/7，均較低。

結論與檢核

最佳k=2，但給定trace前兩人門檻M=3，位置3的6首次過門檻便選取；位置6還有7，所以本次失敗。最佳化的是均勻排列下的成功機率，不是對所有排列都成功。

05-E12 Quicksort 的模型不能偷換

推導

z2..z5共4鍵，最早pivot是兩端時才直接比較，機率2/4=1/2；z1..z6共6鍵，機率2/6=1/3。每對指示相加得到2(7)H6−24=14・49/20−24=103/10。

若全相等且二路分割總把其餘n−1個等值元素送同一側，任意pivot結果都是大小n−1與0，故比較數C(n)=C(n−1)+n−1、C1=0。解n(n−1)/2，n=6為15；隨機選哪個相等身分不改此遞迴。

結論與檢核

相異鍵期望103/10，全等指定二路版本固定15次。漸近全等仍Θ(n^2)，三路等值區處理才會改變模型。

05-E13 公平 bits 產生六種輸出

推導

每輪抽3個bits得z∈{0,⋯,7}，z≤5便回z+1，否則重抽。每輪各合法輸出機率1/8，拒絕率1/4，所以最終機率Σr≥0(1/4)^r(1/8)=1/6。輪數期望4/3，每輪恰3bits，期望4bits。

若直接mod6，輸出1與2各有前像{0,6}、{1,7}，其餘各一個；機率依序1/4、1/4、1/8、1/8、1/8、1/8。

05 機率分析與隨機演算法 • 獨立詳解 （續）

結論與檢核

拒絕抽樣恰均勻，期望4bits，且無限拒絕機率limr→∞(1/4)^r=0。有限期望不是有固定最壞輪數。

05-E14 未知偏硬幣抽出公平位元

推導

每輪獨立擲兩次；HT回0、TH回1，HH或TT重試。兩種接受結果機率同為p(1−p)，接受率q=2p(1−p)>0。最終任一輸出的機率Σr≥0(1−q)^r p(1−p)=1/2，無限重試機率0。

輪數期望1/q，每輪兩次，故期望擲幣次數1/[p(1−p)]。這個等式需要每次硬幣獨立且使用同一p；若只保證每次邊際為p，HT與TH的結構可能改變。

結論與檢核

對每個固定0<p<1，期望有限；當p→0或1時無界，因此沒有對所有p統一的常數成本界。p=0、1時永遠不接受，已由題目前提排除。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

05 機率分析與隨機演算法 • 獨立詳解 (續)

原創驗算程式附錄: probability_depth_checks.py

```
"""Exact finite probability models for Chapter 5 examples; no simulation claims."""
from fractions import Fraction as F
from itertools import product,permutations,combinations
from math import factorial,comb,ceil,log2
from collections import Counter
from functools import lru_cache

def harmonic(n):return sum((F(1,i) for i in range(1,n+1)),F(0))
def shuffle_path(a,choices,reverse=False):
    a=list(a)
    for i,j in zip(range(len(a)-1,0,-1) if reverse else
range(len(a)),choices):a[i],a[j]=a[j],a[i]
    return tuple(a)
def records(a):
    highest=-1;out=[]
    for i,v in enumerate(a):
        if v>highest:out.append(i);highest=v
    return out
def online(a,k):
    threshold=max(a[:k])
    return next((i for i in range(k,len(a)) if a[i]>threshold),len(a)-1)
def longest(a):
    best=current=0
    for v in a:
        current=current+1 if v else 0;best=max(best,current)
    return best

def check():
    shuffle_cases=0;hiring_cases=0;online_cases=0
    for n in range(9):
        counts=Counter(shuffle_path(range(n),js) for js in product(*(range(i,n) for i in
range(n))))
        assert len(counts)==factorial(n) and set(counts.values())==
{1};shuffle_cases+=sum(counts.values())
        rev=Counter(shuffle_path(range(n),js,True) for js in product(*(range(i+1) for i in
range(n-1,0,-1))))
        assert counts==rev
        hits=[0]*n;hist=Counter();wins=[0]*n
        for a in permutations(range(n)):
            rec=records(a)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

05 機率分析與隨機演算法 • 獨立詳解 (續)

```
            for i in rec:hits[i]+=1
            hist[len(rec)]+=1;hiring_cases+=1
            for k in range(1,n):
                wins[k]+=a[online(a,k)]==n-1;online_cases+=1
        if n:
            assert sum(F(c,factorial(n)) for c in hits)==harmonic(n)
            assert all(F(c,factorial(n))==F(1,i+1) for i,c in enumerate(hits))
            assert hist[1]==factorial(n-1) and hist[n]==1
            if n>=2:assert F(hist[2],factorial(n))==harmonic(n-1)/n
        for k in range(1,n):assert F(wins[k],factorial(n))==F(k,n)*(harmonic(n-1)-harmonic(k-
1))
    assert online([2,5,1,4,6,3],2)==4
    assert online([3,1,6,2,4,7,5],2)==2
    assert max((F(k,7)*(harmonic(6)-harmonic(k-1)),k) for k in range(1,7))==(F(29,70),2)
    naive=Counter()
    for js in product(range(3),repeat=3):
        a=list(range(3))
        for i,j in enumerate(js):a[i],a[j]=a[j],a[i]
        naive[tuple(a)]+=1
    assert set(naive.values())=={4,5}
    priorities=Counter()
    for a,b in product(range(4),repeat=2):priorities[a<=b]+=1
    assert priorities=={True:10,False:6}
    birthday_cases=0
    for n in range(1,6):
        for k in range(6):
            collisions=0;pairs=0
            for a in product(range(n),repeat=k):
                collisions+=len(set(a))<k;pairs+=sum(a[i]==a[j] for i,j in
combinations(range(k),2));birthday_cases+=1
            total=n**k;distinct=F(1)
            if k<=n:
                for j in range(k):distinct*=F(n-j,n)
            else:distinct=F(0)
            assert F(collisions,total)==1-distinct
            assert F(pairs,total)==F(comb(k,2),n)
    q=F(1)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

05 機率分析與隨機演算法 • 獨立詳解 (續)

```
    for j in range(5):q*=F(16-j,16)
    assert 1-q==F(4097,8192)
    ap=[(u,v,(2*u-v)%3) for u,v in product(range(3),repeat=2)]
    for i,j in combinations(range(3),2):assert len({(a[i],a[j]) for a in ap})==9
    assert sum(len(set(a))<3 for a in ap)==3
    weighted_collision=weighted_pairs=F(0);probs=[F(1,2),F(1,4),F(1,8),F(1,8)]
    for a in product(range(4),repeat=3):
        p=probs[a[0]]*probs[a[1]]*probs[a[2]]
        weighted_collision+=p*(len(set(a))<3);weighted_pairs+=p*sum(a[i]==a[j] for i,j in
combinations(range(3),2))
    assert weighted_collision==F(95,128) and weighted_pairs==F(33,32)
    balls=0
    for b in range(1,5):
        for t in range(6):
            empty=single=covered=0
            for a in product(range(b),repeat=t):
                loads=Counter(a);empty+=b-len(loads);single+=sum(v==1 for v in
loads.values());covered+=len(loads)==b;balls+=1
            assert F(empty,b**t)==b*F(b-1,b)**t
            expected_single=t*F(b-1,b)**(t-1) if t else F(0)
            assert F(single,b**t)==expected_single
            if (b,t)==(3,4):assert F(covered,b**t)==F(4,9)
        E=F(0)
        for i in range(b-1,-1,-1):E=(1+F(b-i,b)*E)/(1-F(i,b))
        assert E==b*harmonic(b)
    streak_cases=0
    for n in range(13):
        values=[];windows=[0]*(n+1)
        for a in product([0,1],repeat=n):
            L=longest(a);values.append(L);streak_cases+=1
            for k in range(1,n+1):windows[k]+=sum(all(a[i:i+k]) for i in range(n-k+1))
        for k in range(1,n+1):assert F(windows[k],2**n)==F(n-k+1,2**k)
        if n:
            E=F(sum(values),2**n)
            assert E==sum((F(sum(v>=k for v in values),2**n) for k in range(1,n+1)),F(0))
            assert E<=ceil(log2(n))+1
        if n==8:assert sum(v>=3 for v in values)==107
    seq=[c=='H' for c in 'HHHTHHHHTTHTHHTH']
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

05 機率分析與隨機演算法 · 獨立詳解 （續）

```
    assert longest(seq)==4 and [i+1 for i in range(14) if all(seq[i:i+3])]==[1,5,6]
    @lru_cache(None)
    def trees(n):
        if n==1:return ((1,0,F(1)),)
        out=[]
        for p in range(1,n):
            for la,ca,pa in trees(p):
                for lb,cb,pb in trees(n-p):out.append((la+lb,n+ca+cb,pa*pb/F(n-1)))
        return tuple(out)
    tree_cases=0
    for n in range(1,10):
        cases=trees(n);assert sum(p for l,c,p in cases)==1
        assert all(l==n for l,c,p in cases)
        assert sum(c*p for l,c,p in cases)==2*n*(harmonic(n)-1);tree_cases+=len(cases)
    quick_cases=0
    for n in range(1,8):
        total=0
        for order in permutations(range(n)):
            priority={v:i for i,v in enumerate(order)}
            def cost(keys):
                if len(keys)<=1:return 0
                p=min(keys,key=priority.get)
                return len(keys)-1+cost([v for v in keys if v<p])+cost([v for v in keys if
v>p])
            total+=cost(list(range(n)));quick_cases+=1
        assert F(total,factorial(n))==2*(n+1)*harmonic(n)-4*n
    for m in range(1,34):
        k=(m-1).bit_length();accepted=[z for z in range(2**k) if z<m]
        assert len(accepted)==m and 2**k<2*m
    return
{'forward_and_reverse_shuffle_paths':shuffle_cases,'hiring_rank_permutations':hiring_cases,'o
nline_threshold_permutation_cases':online_cases,'birthday_assignments':birthday_cases,'indepe
ndent_ball_assignments':balls,'binary_streak_sequences':streak_cases,'random_split_weighted_t
rees':tree_cases,'uniform_pivot_priority_orders':quick_cases,'six_person_online_success_k2':'
77/180','seven_person_optimum':{'k':2,'success':'29/70'},'nonuniform_birthday_example':
{'collision_probability':'95/128','pair_expectation':'33/32'},'scope':'Exact finite models
validate traces and counterexamples; general probability/invariant/asymptotic proofs are in
the canonical chapter.'}

if __name__=='__main__':
    import json
    print(json.dumps(check(),ensure_ascii=False,indent=2))
```

05 機率分析與隨機演算法 • 獨立詳解 (續)

原創驗算程式附錄: random_partition.py

```
"""Exact recurrence checks for random recursive splits and quicksort."""

from fractions import Fraction


def random_split_expectations(n):
    leaves = [Fraction(0)] * (n + 1)
    stars = [Fraction(0)] * (n + 1)
    leaves[1] = Fraction(1)
    for size in range(2, n + 1):
        leaves[size] = Fraction(2, size - 1) * sum(leaves[1:size])
        stars[size] = size + Fraction(2, size - 1) * sum(stars[1:size])
    return leaves, stars


def harmonic(n):
    return sum((Fraction(1, k) for k in range(1, n + 1)), Fraction(0))


def quicksort_comparisons(n):
    return sum(
        (Fraction(2, distance + 1) * (n - distance) for distance in range(1, n)),
        Fraction(0),
    )

def online_success(k, n):
    return Fraction(k, n) * sum(
        (Fraction(1, i) for i in range(k, n)), Fraction(0)
    )


def expected_longest_head_run(n):
    total = 0
    for bits in range(1 << n):
        longest = run = 0
        for position in range(n):
            if bits >> position & 1:
                run += 1
                longest = max(longest, run)
            else:
                run = 0
        total += longest
    return Fraction(total, 1 << n)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

05 機率分析與隨機演算法 · 獨立詳解 （續）

```
def main():
    maximum = 12
    leaves, stars = random_split_expectations(maximum)
    for n in range(1, maximum + 1):
        assert leaves[n] == n
        assert stars[n] == n * (2 * harmonic(n) - 2)

    rows = []
    for n in range(1, 9):
        rows.append(
            (
                n,
                leaves[n],
                stars[n],
                quicksort_comparisons(n),
            )
        )
    assert rows[4][1:] == (
        Fraction(5),
        Fraction(77, 6),
        Fraction(37, 5),
    )
    assert stars[2] == 2 and stars[3] == 5
    best_k = max(range(1, 100), key=lambda k: online_success(k, 100))
    assert online_success(best_k, 100) > Fraction(1, 3)
    streak_rows = [
        (n, expected_longest_head_run(n)) for n in range(1, 11)
    ]
    assert all(current >= previous for (_, current), (_, previous) in zip(streak_rows[1:],
streak_rows))
    print(
        "PASS random_split_rows="
        + repr([tuple(map(str, row)) for row in rows])
        + f" online_k_100={best_k}"
        + f" online_success_100={float(online_success(best_k, 100)):.6f}"
        + " longest_runs="
        + repr([(n, str(value)) for n, value in streak_rows])
    )


if __name__ == "__main__":
    main()
```

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study